Meeting Analyzer

Detailed NLP Pipeline

An implementation-grounded guide to the audio, speech, NLP, semantic extraction, evidence validation, persistence, and reporting workflow.

Repository: NLP | Generated from the current backend services
Pipeline order is preserved from upload to dashboard/PDF export.

1. Executive overview

Meeting Analyzer converts an existing audio or video recording into a timestamped transcript and a structured meeting report. The design is hybrid: deterministic local components handle media normalization, speech recognition, linguistic preprocessing, classification, topic signals, and extractive summarization; structured semantic extraction adds cross-turn interpretation while strict validation prevents unsupported insights from reaching the dashboard.

The pipeline is not a live transcription system. It processes an uploaded recording, retains the original transcript evidence, and exposes the result through the FastAPI backend for the React dashboard and PDF report.

Canonical processing order

1. Upload recording	**2. Validate** type and storage	**3. Audio** FFmpeg	**4. ASR** Faster-Whisper	**5. NLP** spaCy + NLTK
6. Signals NER | classifier | topics	**7. Intelligence** actions | decisions | questions	**8. Summary** extractive / optional draft	**9. Persist** local + Supabase	**10. Present** dashboard / PDF

2. Pipeline stage reference

Stage	Implementation	Input	Output / guarantee
Upload and validation	routes/upload.py; meeting_store.py	Audio/video file	Stored meeting ID; supported media and bounded file checks.
Audio normalization	services/audio.py	Original media	Mono, 16 kHz PCM WAV produced by FFmpeg.
Transcription	services/transcription.py	Normalized WAV	Segments with start/end seconds, text, language, duration.
Preprocessing	services/preprocessing.py	Full transcript	spaCy Doc, sentences, tokens, filtered tokens, lemmas.
NER	services/ner.py + EntityRuler	spaCy Doc	Named people, organizations, dates, locations, products/technologies.
Classification	services/classification.py	Sentences	Five dialogue-act labels plus model confidence.
Topics	services/topics.py	Sentences / Doc	TF-IDF keywords, LDA or NMF clusters, noun-phrase topics.
Semantic extraction	services/semantic.py	Transcript units	Structured summary, actions, decisions, questions, topics and entities.
Validation/reconciliation	services/intelligence.py; consistency.py	Candidate report + evidence	Only grounded items persist; stable IDs and edits survive re-analysis.
Summarization	services/summarization.py	Transcript	Chronological TF-IDF/MMR excerpts; optional BART/T5 draft.

3. Media ingestion and audio preparation

The upload route stores the original recording under a meeting-specific directory. The audio service accepts common audio extensions (MP3, WAV, FLAC, OGG, M4A) and video containers (MP4, MOV, MKV, AVI, WEBM). Every input is passed through the same normalization command so downstream speech recognition sees a consistent signal.

FFmpeg drops video, mixes channels to mono, resamples to 16 kHz, and writes signed 16-bit PCM WAV. Missing FFmpeg, unsupported extensions, nonexistent paths, and a ten-minute conversion timeout are explicit errors rather than

silent fallbacks.

```
Command contract: ffmpeg -y -i INPUT -vn -ac 1 -ar 16000 -acodec pcm_s16le OUTPUT.wav
```

4. Speech-to-text and transcript representation

Faster-Whisper is loaded lazily and cached per worker. The model size, device, and compute type are configurable through WHISPER_MODEL_SIZE, WHISPER_DEVICE, and WHISPER_COMPUTE_TYPE; defaults are base, CPU, and int8. Transcription uses beam size 5 and voice-activity detection with a 500 ms minimum silence duration.

Field	Meaning	Why it matters
segments[].start / end	Rounded time boundaries in seconds.	Enables source navigation and evidence timestamps.
segments[].text	Text for one detected speech segment.	Preserves the smallest source unit used by later validation.
full_text	Segments joined by spaces.	Canonical text for NLP, hashes, and quote matching.
language	Detected or requested language code.	Controls analysis metadata and reproducibility.
duration	Total media duration in seconds.	Displayed as meeting metadata and used in reports.

5. Linguistic preprocessing

Preprocessing uses one spaCy document so sentence boundaries, token attributes, named entities, and dependency information can be reused. The original text is retained for evidence: normalization is intentionally light and does not replace the source transcript.

Processing sequence

1) spaCy loads en_core_web_sm and adds an EntityRuler; 2) spaCy performs sentence segmentation and tokenization; 3) stop words are the union of spaCy and NLTK English stop words; 4) punctuation, spaces, and stop words are excluded from clean tokens; 5) surviving tokens are lower-cased lemmas for downstream statistical features.

The preprocessing response contains sentences, raw tokens, clean_tokens, lemmas, and the spaCy Doc. A transcript exceeding spaCy's max_length raises an explicit error and must be split before analysis.

6. Entity extraction

The NER layer combines spaCy's statistical recognizer with deterministic EntityRuler patterns for technologies and project identifiers. The implemented report contract groups normalized entities into people, organizations, dates, locations, and products. Technology and project patterns cover terms such as Python, React, FastAPI, Whisper, NLP, phase 2, sprint 3, and version 1.

Entities are signals, not evidence by themselves. A report item still needs a verbatim transcript quote and valid segment mapping before it is accepted into meeting intelligence.

7. Dialogue-act classification

Each sentence is classified as ACTION ITEM, DECISION, QUESTION, DISCUSSION, or INFORMATION. The baseline is a persisted scikit-learn Pipeline trained from data/training_sentences.csv. It uses word TF-IDF features with unigram-to-trigram ranges, up to 20,000 features, sublinear term frequency, and balanced multinomial Logistic Regression.

The returned confidence is the maximum class probability and is useful for inspection, but it is not a calibrated factuality score. High-level rule checks then promote explicit decisions, action patterns, and questions when the sentence contains stronger evidence than the model label alone. A Hugging Face classifier can replace the internals later without changing the public result shape.

Label	Interpretation	Downstream use
ACTION ITEM	Explicit assignment, commitment, request, or agreed next step.	Action extraction and status/detail inference.
DECISION	Settled choice, approval, rejection, or selected direction.	Decision statements.
QUESTION	Substantive uncertainty or unresolved issue.	Question resolution tracking.
DISCUSSION	Meaningful discussion without a detected commitment.	Topics and summary context.
INFORMATION	Context or factual statement.	Context and summary candidate.

8. Topic discovery

Topic extraction has two complementary levels. Corpus-level TF-IDF ranks unigrams and bigrams after a meeting-specific stop-word set. LDA uses count vectors for probabilistic latent topics; NMF is available as an alternative over TF-IDF vectors and is often sharper for short documents. Topic count is capped by the number of sentences and vocabulary size, and a fixed random_state of 42 supports repeatability.

The service also ranks noun phrases from the spaCy Doc. These phrases remain transcript-derived discussion labels, while the semantic report may provide broader labels only when each label has supporting evidence.

9. Grounded meeting intelligence

The semantic stage receives ordered transcript units containing source segment IDs, speaker labels when available, text, and start times. Short meetings use one structured extraction request; long meetings use byte-bounded chunks with adjacent-turn overlap, local extraction, and deterministic global merging. The schema covers summary bullets, key takeaway, actions, follow-ups, decisions, questions, topics, entities, metadata, and limitations.

The provider is instructed to treat transcript content as data, extract all distinct supported items, preserve the original language in evidence, and never invent owners, deadlines, priorities, answers, or participants. A first-person commitment may resolve to the labeled speaker, but speaker identity is not inferred when the recording has no labels.

Object	Required meaning	Evidence rules
Action	task, optional owner/deadline/priority, status (Pending by default)	Exact quote; owner/deadline evidence when contextual.
Decision	settled decision statement	Exact quote and supporting segment IDs.
Question	question, status, optional answer	Unresolved unless answer and answer evidence are grounded.
Topic	label, keywords, optional relevance	Quote must support the topic concept.
Summary	3-6 concise factual bullets	One summary_evidence object per bullet.

10. Evidence validation and reconciliation

Before persistence, every candidate insight is normalized and checked. The quote must be a contiguous substring of the canonical transcript (with a bounded fuzzy-trimming recovery for minor edge-word errors), segment IDs must be valid integers, and the quote must occur within the cited source span. Unique matches receive a timestamp from the first supporting segment and a speaker only when all supporting segments agree.

Deduplication merges paraphrases only when semantic terms and source evidence identify the same event. Actions with conflicting owners or deadlines remain separate. This prevents a broad similarity threshold from collapsing distinct work items.

Re-analysis is controlled by a SHA-256 manifest containing analysis and prompt versions, model/configuration hashes, input hash, schema hash, chunk policy, merge policy, and output settings. Stable IDs are derived from meeting ID,

category, source segment IDs, and a task/decision suffix when multiple insights share a turn. Existing user edits are carried forward only when the underlying task and evidence still match.

Reconciliation state model

complete means a validated report is available; partial means some fields required recovery or review; empty means no usable intelligence was produced. Provider or parsing failure is surfaced as an API failure and saved as a failed attempt rather than converted into a successful empty report.

11. Summarization

The default extractive summary is source-preserving. It removes duplicate sentences and low-information greetings, vectorizes substantive sentences with TF-IDF, scores centroid relevance, boosts actions/decisions and risk signals, then applies maximal marginal relevance so selected sentences are diverse. The final sentences are returned in transcript order.

Optional abstractive summarization uses a Hugging Face BART/T5-compatible pipeline on CPU. Text is split at sentence boundaries under a character budget, each chunk is summarized, and the result is marked as a draft. The shareable report remains grounded in transcript excerpts; an abstractive draft must receive human review.

12. Persistence, API, and presentation

The analysis route loads a stored transcript, builds the current manifest, uses a cache hit when the prior complete report has the same pipeline and input identity, runs analysis under a meeting lock, checkpoints long analysis, reconciles the candidate, and persists the canonical payload. Local JSON storage supports recovery; Supabase stores compatible boundary fields and the canonical intelligence object.

The frontend consumes the report through the existing Overview, Transcript, and Analytics views. Evidence remains navigable to the first matching timestamp when source mapping is unambiguous. PDF export uses ReportLab and should display the evidence-backed report rather than independently re-running extraction.

13. Operational configuration

Area	Configuration	Operational note
ASR	WHISPER_MODEL_SIZE, WHISPER_DEVICE, WHISPER_COMPUTE_TYPE	GPU failure retries once on CPU; model is lazy-loaded.
Semantic provider	GROQ_API_KEY / MEETING_LLM_API_KEY and MEETING_LLM_* settings	Credentials stay server-side; bounded retries handle transient limits.
NLP model	en_core_web_sm plus NLTK stopwords	Install/download resources before first analysis.
Storage	Local meeting files and optional Supabase	No SQL migration is required for the canonical JSONB field.
Validation	backend.scripts.test_report_accuracy and related tests	Regression fixtures are not a production accuracy benchmark.

14. Limitations and evaluation boundaries

The extraction rules favor precision and may omit implicit commitments, unusual task verbs, conditional assignments, and cross-sentence resolutions. Whisper does not perform diarization in the current service, so unlabeled audio cannot reliably identify speakers. English spaCy/NLTK models are used for classical NLP, although semantic extraction can process the original multilingual text without forced translation.

Model probability is not factuality. Fixture-level re-analysis stability does not establish general accuracy, and the regression suite is not a measured production benchmark. Representative annotated recordings are required before treating generated reports as authoritative.

15. Reproducible execution checklist

1) Install Python dependencies and FFmpeg. 2) Download en_core_web_sm and ensure NLTK stopwords are available. 3) Configure server credentials only in .env when semantic extraction is enabled. 4) Start FastAPI and the React/Vite frontend. 5) Upload an existing recording. 6) Inspect the raw timestamped transcript. 7) Run analysis and verify evidence quotes, segment IDs, actions, decisions, questions, and limitations. 8) Re-analyse the same meeting to verify cache/reconciliation behavior. 9) Run the focused regression tests before sharing a report.

Repository implementation map

Concern	Primary files
HTTP orchestration	backend/main.py; backend/routes/upload.py; backend/routes/analysis.py
Media and ASR	backend/services/audio.py; backend/services/transcription.py
Classical NLP	backend/services/preprocessing.py; ner.py; classification.py; topics.py
Semantic intelligence	backend/services/semantic.py; intelligence.py; consistency.py
Summaries	backend/services/summarization.py
Storage and reports	backend/services/meeting_store.py; backend/db.py; routes/report.py
Validation	backend/scripts/test_*.py; data/training_sentences.csv

End of document